PROJECT EXPLAINER / BEGINNER-FRIENDLY GUIDE

SiddAi VideoDost

Paste a video link. Get the transcript, structured notes, and a chatbot grounded in the video.

INPUT	YouTube URL / video link
TRANSCRIPT	Supadata API retrieves text without downloading YouTube audio in the URL path
OUTPUT	Title, summary, action items, decisions, open questions, and transcript chat

Built with Python, Streamlit, Supadata, Groq, Sarvam AI, LangChain, embeddings, and ChromaDB.

LIVE DEMO

https://8thqrtayvgnsman226nljl.streamlit.app/

SOURCE REPOSITORY

https://github.com/siddhant754962/SIddAi-final

Prepared 13 October 2026 from the current SIddAi-final source files.

1. Explain the project in simple words

SiddAi VideoDost is an AI-powered video understanding assistant. A user pastes a YouTube link, and the application tries to obtain the video's transcript through Supadata. It then uses AI to turn that text into structured notes and provides a chat interface for asking questions about the same video.

The problem it solves

- Long lectures and tutorials can take a lot of time to watch and revise.
- Important concepts, tasks, and decisions are difficult to find manually in a long video.
- A generic chatbot may not be grounded in the actual content of a selected video.

What users get

Output	Plain-English meaning
Title	A short title generated from the transcript.
Summary	A readable overview of the central points.
Action items	Tasks or next steps explicitly mentioned in the source.
Key decisions	Important choices or conclusions found in the transcript.
Open questions	Unresolved questions or follow-up topics.
Chat with video	Ask a question and get an answer based on retrieved transcript passages.

One-sentence pitch

SiddAi VideoDost turns a YouTube video's transcript into organized notes and a searchable, transcript-grounded Q&A experience.

Who would use it?

Students revising lectures, developers studying tutorials, professionals reviewing recorded discussions, and anyone who wants to search or question video content more quickly.

2. System architecture

The project is split into a user interface, a coordinator that runs the workflow, services for transcript and language-model calls, and a retrieval layer for question-answering.



Main components

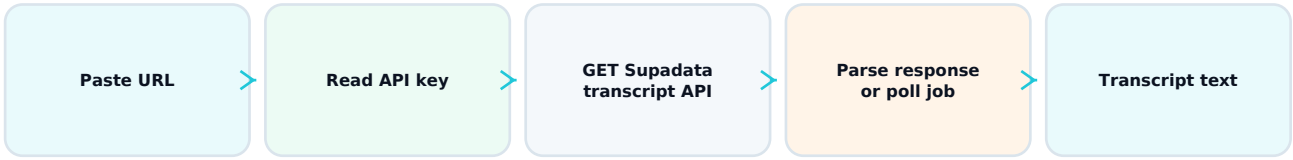
Component	Responsibility	Think of it as...
app.py	Streamlit interface: input field, language choice, process button, results, transcript tab, downloads and chat.	The front desk.
main.py	Reads secrets, obtains transcript, calls AI stages, builds the RAG chain, and returns results.	The workflow manager.
Supadata API	Obtains transcript text for URL-based processing.	The transcript provider.
core/summarizer.py	Uses Groq's language model for title generation and chunk-based summary.	The note writer.
core/extractor.py	Separately extracts action items, decisions and open questions.	The structured analyst.
core/vector_store.py	Splits transcript text, embeds chunks and stores them in ChromaDB.	The searchable library.
core/rag_engine.py	Retrieves relevant chunks and sends them with a question to the LLM.	The video Q&A; brain.
core/transcriber.py + utils/audio_processor.py	Processes local media paths: conversion, chunking and Sarvam speech-to-text.	The audio path.

Important distinction

In the current main.py, HTTP/HTTPS URLs use Supadata. A non-URL source follows the local-media path and uses audio processing plus Sarvam transcription.

3. How a YouTube link is processed

This is the path to explain when someone asks how the app avoids downloading audio from YouTube during normal link-based processing.



What the current code does

- main.py checks the source string. If it begins with http:// or https://, it calls fetch_supadata_transcript().
- The API key is read from SUPADATA_API_KEY in environment variables or Streamlit Secrets.
- The app sends a GET request to https://api.supadata.ai/v1/transcript with the URL, text=true, mode=auto, and an x-api-key header.
- If the provider returns HTTP 202, the code reads a jobId and polls the transcript job endpoint every two seconds for up to 90 attempts.
- The response parser reads transcript content supplied as a string or as a list of text items.
- If the final transcript is empty, missing or inaccessible, the pipeline raises an error instead of silently inventing a transcript.

Why use this architecture?

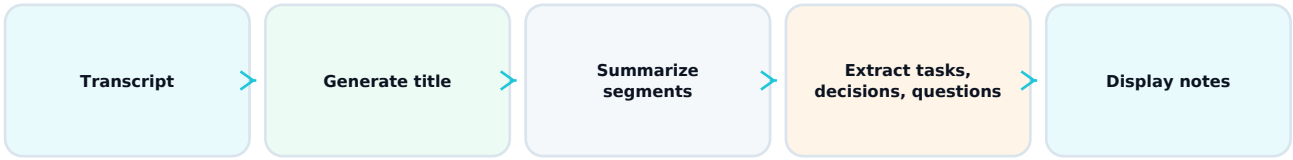
It separates video-source access from downstream AI processing. The application requests transcript text from a dedicated provider instead of relying on the Streamlit server to download YouTube audio. This avoids the specific audio-download path for URLs, but does not guarantee that every video is available from the provider.

Do not overclaim

A transcript provider is a dependency, not a promise that every URL will work. Captions, provider policy, quotas, video privacy and API errors can still affect retrieval.

4. How the transcript becomes useful notes

Once the transcript is available, the project uses a series of focused LLM calls rather than asking one prompt to do every task at once.



Title and summary - core/summarizer.py

The code uses LangChain's ChatGroq integration with model name openai/gpt-oss-20b. For a long transcript, RecursiveCharacterTextSplitter divides text into chunks of about 2,000 characters with 150 characters of overlap. Each part is summarized, then a second prompt combines the partial summaries into a coherent final summary. A separate prompt creates a short title from a transcript excerpt.

Structured extraction - core/extractor.py

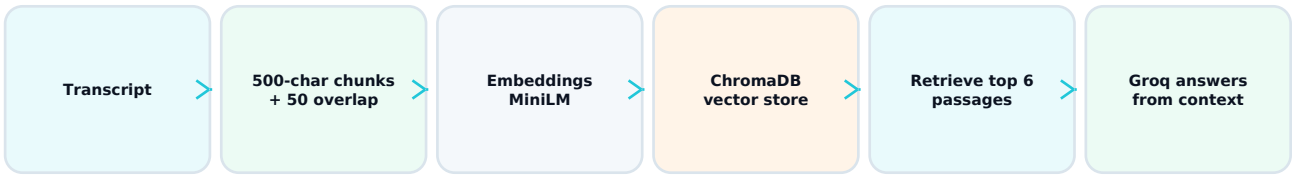
Output	Extraction goal	Guardrail in prompt
Action items	Tasks, owner and deadline when mentioned.	Do not invent tasks, owners or deadlines.
Key decisions	Decisions that were actually agreed upon.	Separate confirmed choices from ideas.
Open questions	Unresolved questions and follow-up topics.	Do not add questions that are not supported by transcript.

The benefit of separate tasks

It makes outputs easier to read and gives each prompt a clear purpose. It does not guarantee perfect accuracy, so critical facts should still be checked against the transcript.

5. RAG: chatting with the video

RAG stands for Retrieval-Augmented Generation. The application searches the transcript for relevant text first, then asks the LLM to answer using that retrieved context. It is different from sending a question to a general chatbot without supplying source evidence.



The current implementation

Setting	Current source-code value
Embedding model	sentence-transformers/all-MiniLM-L6-v2 through HuggingFaceEmbeddings, configured for CPU and normalized embeddings.
Vector store	ChromaDB persistent client using the project vector_db directory.
RAG chunk size	500 characters with 50-character overlap.
Retriever	Similarity search returns k=6 relevant documents.
Answer model	Groq ChatGroq with openai/gpt-oss-20b and temperature 0.0.
Prompt rule	Use only the retrieved transcript excerpts; if the answer is absent, say so instead of inventing information.

Simple analogy

Think of a librarian. First the librarian finds six pages most related to your question. Then a writer reads those pages and explains the answer. The retrieval step is what makes the answer video-specific.

Why embeddings?

Embeddings represent text meaning as numbers. Similar passages can be found even when a question uses different words from the original transcript.

6. Project structure and code roles

Use this page to answer the question: 'Which files did you write, and what does each one do?'

File	Responsibility	When you would inspect it
app.py	Streamlit layout, input control, language selector, processing action, results, transcript view, chat and Markdown downloads.	UI bugs, display changes, chat interaction.
main.py	Supadata API key handling, transcript extraction, async polling, main pipeline, title/summary/extraction calls and RAG construction.	URL processing flow, API errors, pipeline orchestration.
core/summarizer.py	Groq LLM setup, retries on rate-limit errors, transcript splitting, title and summary generation.	Summary quality, model errors, prompt changes.
core/extractor.py	Dedicated LLM prompts for action items, decisions and open questions.	Structured notes quality.
core/rag_engine.py	RAG prompt, retriever wiring, context formatting, question invocation.	Chat answers and grounding.
core/vector_store.py	Video/source IDs, transcript chunking, embeddings, Chroma collection creation and retrieval.	Indexing, search quality, vector-store issues.
core/transcriber.py	Splits prepared audio into 25-second pieces and calls Sarvam speech-to-text.	Local-media transcription errors.
utils/audio_processor.py	Audio conversion to mono 16 kHz WAV and 10-minute chunking; also contains yt-dlp URL download support for that path.	Audio decoding, FFmpeg, local media processing.
requirements.txt	Python package dependencies.	Install/build failures.

Main concept

app.py is the user interface. main.py is the coordinator. The core/ and utils/ modules contain the specialised processing steps.

7. Setup, API keys, and running the app

The project depends on third-party services. The code should read their credentials from environment variables or Streamlit Secrets instead of hard-coding keys in Python files.

Secret	Used for	When needed	Purpose
SUPADATA_API_KEY	main.py	Required for link-based transcript retrieval.	Authenticates transcript API calls.
GROQ_API_KEY	Summarizer, extractor, RAG engine	Required for titles, summaries, extraction and chat.	Authenticates calls to the Groq-hosted LLM.
SARVAM_API_KEY	core/transcriber.py	Required only when the local audio transcription path is used.	Authenticates speech-to-text requests for audio pieces.

Local setup (Windows PowerShell)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
streamlit run app.py
```

Local .env example - placeholders only

```
SUPADATA_API_KEY=your_supadata_key
GROQ_API_KEY=your_groq_key
SARVAM_API_KEY=your_sarvam_key
```

Deployment checklist

- The hosting service must point to the current SIddAi-final repository and use app.py as the entry point.
- Add SUPADATA_API_KEY and GROQ_API_KEY to Streamlit Secrets; add SARVAM_API_KEY if the local-media path is used.
- Keep .env, cookies, tokens and API keys out of GitHub.
- Use requirements.txt for packages and check deploy logs when dependencies change.

Live demo: https://8thqrtayvqnsman226nljl.streamlit.app/

Repository: https://github.com/siddhant754962/SIddAi-final

8. Limitations and what to say honestly

A credible project explanation includes known limits. Do not say the application can process every YouTube URL under all conditions.

Transcript provider dependency

URL-based processing depends on Supadata's transcript endpoint, API key, plan/credits, video accessibility and provider response. The current code handles immediate responses and queued HTTP 202 jobs, but it still has a finite polling timeout. A missing or empty transcript stops the pipeline.

Local media path

The source code can process a local media file path, convert it to WAV and send audio chunks to Sarvam. The current SIddAi-final Streamlit UI exposes a text input for a YouTube URL or local path; it does not currently expose a browser file-uploader widget. A path on a user's personal computer is not automatically accessible to a cloud server.

Language handling

The UI offers English and Hinglish. The Supadata request uses mode=auto and does not currently pass the selected UI language as an API parameter. The local Sarvam transcription code currently sends language_code='hi-IN' and mode='translit', so the selector should not be presented as fully language-aware across every processing path.

RAG and persistence

ChromaDB is configured to store vectors in the local vector_db folder. Depending on the hosting environment, local data may not be persistent across restarts/redeploys unless persistent storage is configured. Results and chat history are held in Streamlit session state during the user's session.

Output accuracy and cost

- Summaries and extracted items can miss context or contain mistakes; verify important details against the transcript.
- Large transcripts may cause multiple LLM calls, increased latency, or provider rate limits.
- Transcript access and AI services have their own usage limits and availability constraints.

Good answer in an interview

I separated ingestion from the AI workflow. For YouTube links, the application uses a hosted transcript provider because direct downloads can be blocked from cloud hosting. The transcript then feeds the same summary, extraction, and RAG pipeline.

9. Interview and demo script

30-second project introduction

SiddAi VideoDost is an AI-powered video understanding tool. A user pastes a YouTube URL, the app retrieves its transcript through Supadata, and then uses a language model to generate a summary, action items, decisions and open questions. I also built a RAG-based chat feature using embeddings and ChromaDB, so a user can ask questions that are answered using relevant parts of the transcript.

90-second technical explanation

The frontend is built using Streamlit. main.py coordinates the workflow and calls the Supadata transcript API for URL inputs. It handles both immediate transcript responses and queued jobs. After transcript extraction, the Groq-hosted openai/gpt-oss-20b model is used through LangChain to generate a title and summary and extract action items, decisions and open questions. For chat, the transcript is split into overlapping chunks, embedded using sentence-transformers/all-MiniLM-L6-v2, and stored in ChromaDB. When the user asks a question, the retriever finds six relevant passages and the RAG prompt asks the LLM to answer from that context. A separate local-media path uses pydub/FFmpeg and Sarvam speech-to-text.

Common interview questions

Question	Answer outline
Why Supadata?	It provides transcript retrieval through a hosted API, avoiding direct YouTube audio download in the URL path.
Why RAG?	It retrieves source passages before generating an answer, helping keep responses connected to the selected video.
Why embeddings?	They represent text numerically so semantically relevant passages can be found even when wording differs.
Why chunking?	It keeps inputs manageable for summarization and improves focused retrieval.
What are the current limits?	Transcript-provider access/quotas, LLM rate limits, local storage persistence, and language-handling differences for local audio.
What would you improve next?	Add a true browser file upload for local media, make ASR language selection accurate, add durable storage, time-stamped source citations, and end-to-end tests.

Demo order

Open the live app -> paste a public YouTube link -> process transcript -> show the summary and extracted notes -> open the transcript -> ask a specific question -> explain that RAG retrieves relevant transcript chunks before generating the answer.

10. Quick glossary

Term	Simple definition	In this project
API	A way for software services to communicate.	The app calls Supadata, Groq and optionally Sarvam.
Transcript	Written form of the words spoken in a video.	Input to the note-generation and RAG steps.
LLM	A model that reads and generates text.	Groq hosts the model used for summaries, extraction and answers.
Prompt	Instructions that define what the model should do.	Prompts shape the summary, extraction and chat behavior.
Chunk	A smaller piece of a long transcript or audio file.	Text chunks are used for summary/retrieval; audio is chunked for ASR.
Embedding	Numbers representing aspects of text meaning.	MiniLM embeddings support semantic search.
Vector database	A store designed for similarity search over vectors.	ChromaDB stores transcript chunks and their embeddings.
Retriever	A component that searches for relevant pieces of information.	Returns six transcript chunks for a question.
RAG	Retrieval-Augmented Generation: search first, generate second.	Anchors the video chatbot to transcript excerpts.
Secret / API key	A credential that grants access to a service.	Stored in .env locally or in Streamlit Secrets in deployment.

The easiest way to remember the system



Repository: https://github.com/siddhant754962/SIddAi-final

Live app: https://8thqrtayvqnsman226nljl.streamlit.app/

This guide is based on the SIddAi-final README and main source files checked on 13 October 2026. The app URL is the one supplied by the project owner; actual runtime availability, API access and quota depend on the live deployment and provider settings.